

Module 1

FUNDAMENTALS OF COMPUTER SCIENCE. COMPUTER SOFTWARE

Unit 1. Fundamentals of Computer Science

Exercise 1.1. *Study the vocabulary to text 1.1.*

engineering activities – конструкторська робота

theoretical activities – теоретична діяльність

design – розробка - design

hardware – апаратне забезпечення

software – програмне забезпечення (ПЗ)

encompass – охоплювати

performance studies – дослідження експлуатаційних характеристик

technique – метод

queueing theory – теорія черг

estimation – оцінка

reliability – надійність

availability – експлуатаційна готовність, працездатність

complicated – складний

experimentation – експериментування, проведення експериментів

incorporate – включати (*до свого складу*)

development cycle – цикл розробки

overlap – перекривати

instruction – команда

carry out – виконувати

software engineering – програмотехніка, інженерія розроблення ПЗ

artificial intelligence – штучний інтелект

date to – датуватися

numerical ana-lysis – числовий аналіз

digital computer – цифровий комп'ютер

electrical engineering – електротехніка

primarily – передусім

basics – основи

circuit – схема (*електрична, електронна*)

input – 1) *n* вхід; 2) *n* введення; 3) *v* вводити

output – 1) *n* вихід; 2) *n* виведення; 3) *v* виводити

arbitrary – довільний

storage – зберігання

represent – представляти

device – пристрій

binary digit – двійкова цифра

transmission – передача

notational system – система позначень

yield – давати

calculus – числення

interpretable symbol – інтерпретовний символ

employ – застосовувати, використовувати

gate – вентиль, логічний елемент

needed – необхідний, потрібний

extension – розширення, доповнення

milestone – віха

specification – *тут* деталізація, уточнення

infinite – нескінченний

proof– випробування

breakthrough – прорив

access – доступ

execution – виконання

critical – важливий

approach – підхід

Exercise 1.2. *Learn the following speech patterns.*

A

He did his job **as well as** we expected.

Professor Hunt teaches programming **as well as** mathematics.

B

None of us knew **whether or not** there would be a meeting.

You'll stay here **whether** you like it **or not**.

C

Some people do not have confidence **when using** computers.

When shaped the diamond should be faceted.

D

It was they/them who told me the news.

It is in the memory that the data are stored

E

Supercomputers are very expensive **such that** ordinary customers cannot afford them.

A proper fraction is a vulgar fraction **such that** its numerator is smaller than the denominator.

F

He didn't show a flicker of interest **in what** I was saying to him.

He asked me for my advice **on what** he should do.

G

Design variables are often bounded, **that is**, they have maximum and minimum values.

Teleconferencing systems allow teachers to communicate with students in real time, **that is**, with no delays in the transmissions.

Exercise 1.3. *Read and translate text 1.1.*

TEXT 1.1. BASICS OF COMPUTER SCIENCE

The field of computer science includes engineering activities such as the design of computers and of the hardware and software that make up computer systems. It also encompasses theoretical, mathematical activities, such as the design and analysis of algorithms, performance studies of systems and their components by means of techniques like queueing theory, and the estimation of the reliability and availability of systems by probabilistic techniques. Since computer systems are often too large and complicated to allow a designer to predict failure or success without testing, experimentation is incorporated into the development cycle. Computer science is generally considered a discipline separate from computer engineering, although the two disciplines overlap extensively in the area of computer architecture, which is the design and study of computer systems.

The major subdisciplines of computer science have traditionally been:

- 1) architecture (including all levels of hardware design, as well as the integration of hardware and software components to form computer systems);
- 2) software (the programs, or sets of instructions, that tell a computer how to carry out tasks), here subdivided into software engineering, programming languages, operating systems, information systems and databases, artificial intelligence, and computer graphics;
- 3) theory, which includes computational methods and numerical analysis on the one hand and data structures and algorithms on the other.

Computer science as an independent discipline dates to only about 1960, although the electronic digital computer that is the object of its study was invented some two decades earlier. The roots of computer science lie primarily in the related fields of electrical engineering and mathematics. Electrical engineering provides the basics of circuit design – namely, the idea that electrical impulses input to a circuit can be combined to produce arbitrary outputs. The invention of the transistor and the miniaturization of circuits, along with the invention of electronic, magnetic, and optical media for the storage of information, resulted from advances in electrical engineering and physics. Mathematics is the source of one of the key concepts in the development of the computer – the idea that all information can be represented as sequences of zeros and ones. In the binary number system numbers are represented by a sequence of the binary digits 0 and 1 in the same way that numbers in

the familiar decimal system are represented using the digits 0 through 9. The relative ease with which two states (e.g., high and low voltage) can be realized in electrical and electronic devices led naturally to the binary digit, or bit, becoming the basic unit of data storage and transmission in a computer system.

Boolean algebra, a notational system developed in the 19th century by an English mathematician George Boole, supplied formalism for designing a circuit with binary input values of 0s and 1s (false or true, respectively, in the terminology of logic) to yield any desired combination of 0s and 1s as output. One of the primary requirements when dealing with digital circuits is to find ways to make them as simple as possible. This constantly requires that complex logical expressions be reduced to simpler expressions that nevertheless produce the same result. Boolean algebra permits an algebraic manipulation of logical statements that can demonstrate whether or not a statement is true and show how a complicated statement can be rephrased in a simpler, more convenient form without changing its meaning. George Boole believed in what he called the 'process of analysis', that is, the process by which combinations of interpretable symbols are obtained. It is the use of these symbols according to well-determined methods of combination that he believed presented 'true calculus'. Today, all our components employ Boole's logic system – using microchips that contain thousands of tiny electronic switches arranged into logical gates that produce predictable and reliable conclusions. A gate is an electronic circuit such that its output is fully determined by the state of its inputs.

Theoretical work on computability, which began in the 1930s, provided the needed extension to the design of whole machines. A milestone was the 1936 specification of the conceptual Turing machine (a theoretical device that manipulates an infinite string of 0s and 1s) by the British mathematician Alan Turing and his proof of the model's computational power. Another breakthrough was the concept of the stored-program computer, usually credited to the Hungarian-American mathematician John von Neumann. This idea – that instructions as well as data should be stored in the computer's memory for fast access and execution – was critical to the development of the modern computer. Previous thinking was limited to the calculator approach, in which instructions are entered one at a time.

Exercise 1.4. *Find in text 1.1 the English for:*

зберігання інформації; база даних; обчислювальні методи; один за одним; мова програмування; споріднені області; цифри від 0 до 9; електротехніка; процес (цикл) розробки; комп'ютерна графіка; приписувати що-небудь кому-небудь (вважати що-небудь чийось заслугою); дослідження експлуатаційних характеристик систем; теорія черг; відносна простота; булева алгебра; структури даних; забезпечити формалізацію; успіхи в розвитку електротехніки; обчислювальна техніка; передбачити відмову; магнітні та оптичні носії; значною мірою перекриватися; так само, як; схемотехніка; штучний інтелект; з одного боку, ... з іншого боку; методи теорії ймовірностей; числовий аналіз; поряд із; бути (стати) наслідком чого-небудь; двійкова система числення; операційна система; система позначень; інженерія розроблення програмного забезпечення.

Exercise 1.5. *Translate into English paying special attention to the italicized words.*

1. **Двійкова система числення** базується на застосуванні двох цифр – 0 та 1, тоді як основу десяткової системи становлять **цифри від 0 до 9**. 2. Особливістю багатьох систем **штучного інтелекту** є те, що **поряд із базою даних** вони мають базу знань. 3. **Відносна простота** спілкування з сучасним комп'ютером значною мірою є **наслідком** досконало-сті його **операційної системи**. 4. **З одного боку, успіхи в розвитку електротехніки** сприяли мініатюризації апаратного забезпечення комп'ютерів, а **з іншого** – математика запропонувала нову, **двійкову систему числення**, що стала основою цифрової **схемотехніки**. 5. **Булева алгебра**, що була створена в XIX сторіччі, описує операції над числами в **двійковій системі числення**. 6. **Інженерія розроблення програмного забезпечення** займається питаннями розробки і створення **мов програмування, операційних систем, баз та структур даних** та багатьма іншими пи-таннями. 7. **Обчислювальні методи**, що засто-совуються для оброблення двійкової інформації в цифрових комп'ютерах, базую-ться на законах **булевої алгебри**. 8. Створення першого вітчизняного комп'ютера, яке датується 1951 роком, **вважається заслугою** київської наукової групи, яку очолював академік Лебедев. 9. Ознакою часу є застосування зараз **оптичних носіїв** для **зберігання інформації** у комп'ютерах, інформаційних системах та в багатьох інших **споріднених галузях**. 10. Булева алгебра **забезпечила формалізацію** цифрової **схемотехніки так само, як** диференційні рівняння стали математичною основою створення аналогових обчислювальних систем. 11. Теорія нечітких множин (fuzzy-set theory) **забезпечує формалізацію** представлення нечітких значень (uncertain values) фізичних величин. 12. У цій експертній системі правила перевіряються **одне за одним** доки одне з них не спрацює. 13. У **двійковій системі числення** ми користуємося двома цифрами – 0 та 1 **так само, як** ми використовуємо **цифри від 0 до 9** в десятковій системі. 14. **Магнітні та оптичні носії** застосовуються для **зберігання інформації** в сучасних комп'ютерних системах. 15. У наш час засоби **штучного інтелекту** широко застосовуються в управлінні технологічними процесами, діагностиці несправностей, а також в інших **споріднених галузях**.

Exercise 1.6. Say whether the following sentences are true or false. Correct the false ones.

1. The field of computer science includes engineering activities such as performance studies of systems, numerical analysis and artificial intelligence. 2. Since computer systems are often too large and complicated to allow a designer to predict failure or success without testing, the estimation of their reliability is incorporated into the development cycle. 3. Theoretical work on computability, which began in the 1930s, provided the needed extension to the design of whole machines. 4. The major subdisciplines of computer science have traditionally been architecture, software, and computer security. 5. Architecture includes all levels of hardware design, as well as the integration of hardware and software components to form computer systems. 6. Software includes algorithms that tell a computer how to carry out tasks. 7. Theory includes computational methods and numerical analysis on the one hand and data structures and algorithms on the other. 8. Computer science as an independent discipline dates to only about 1950, although the electronic digital computer that is the object of its study was invented some two decades later. 9. Mathematics is the source of one of the key concepts in the development of the computer – the idea that all information can be represented as sequences of zeros and ones. 10. John von Neumann's idea was that instructions as well as data should be entered into the computer one at a time.

Exercise 1.7. Complete the sentences translating their Ukrainian parts into English.

1. The field of computer science also encompasses theoretical activities, such as (розробка та аналіз алгоритмів, дослідження експлуатаційних характеристик систем та їхніх компонентів за допомогою методів, таких як теорія чепр). 2. Computer science is generally considered a discipline separate from computer engineering, (хоча ці дві дисципліни значною мірою перекриваються в галузі комп'ютерної архітектури, якою є розробка та дослідження комп'ютерних систем). 3. (Електротехніка стала джерелом основ схемотехніки, а саме), the idea that electrical impulses input to a circuit can be combined to produce arbitrary outputs. 4. The invention of the transistor and the miniaturization of circuits, (поряд із винайденням електронних, магнітних та оптичних носіїв для зберігання інформації стали наслідком досягнень в електротехніці та фізиці). 5. (Відносна простота, з якою два стани – наприклад, високий та низький рівні напруги – можна реалізувати в електронних пристроях), led naturally to the binary digit, or bit. 6. (Джордж Буль вірив у те, що він називав “процесом аналізу”, тобто) the process by which combinations of interpretable symbols are obtained. 7. (Саме використання цих символів) according to well-determined methods of combination (він вважав “справжнім численням”). 8. A milestone was the 1936 specification of the conceptual Turing machine (теоретичного пристрою, який оперує нескінченною послідовністю нулів та одиниць).

Exercise 1.8. Translate into English.

1. Поле діяльності комп'ютерних наук також охоплює оцінку надійності та працездатності систем методами теорії ймовірностей. 2. Корені обчислювальної техніки лежать переважно у споріднених галузях електротехніки та математики. 3. У двійковій системі числення числа представлені послідовністю двійкових цифр 0 та 1 так само, як числа у знайомій десятковій системі представлені цифрами від 0 до 9. 4. Булева алгебра забезпечила формалізацію для розробки схем із двійковими вхідними величинами. 5. Логічний елемент – це електронна схема, така, що її вихід повністю визначається станом її входів. 6. Ще одним проривом стала концепція комп'ютера з програмою, що зберігається у пам'яті, автором якої вважають угорсько-американського математика Джона фон Неймана. 7. Булева алгебра робить можливими алгебраїчні операції над логічними твердженнями, які показують, чи є твердження істинним, чи ні. 8. Однією з основних вимог під час роботи з цифровими схемами є необхідність знаходження способів зробити їх якомога простішими.

Exercise 1.9. Answer the questions.

1. What does the field of computer science include? 2. Why is experimentation incorporated into the computer system development cycle? 3. What can you say about the interconnection of computer science and computer engineering? Is computer science a distinct field of knowledge? 4. Could you outline the major subdisciplines of computer science? 5. What fields of knowledge do the roots of computer science lie in? 6. What did advances in electrical engineering and physics result in? 7. Why was the binary number system chosen for representing data in the digital computer? 8. What is Boolean algebra? What was it designed for? 9. What does Boolean algebra permit by algebraic manipulations of logical statements? 10. How is Boole's logic system employed in the computer engineering of today? What is a gate? 11. What was the merit of Alan Turing? And that of John von Neumann? 12. What is the importance of Neumann's idea?

Exercise 1.10. Give derivatives of the following words and explain their meanings.

Compute, include, system, form, method, structure, number, sequence, digit, object, develop, base, provide, supply, value, simple, complex, reduce, produce, relate, result, permit, demonstrate, complicate, mean, process, change, use, employ, determine, access, modern, limit.

Exercise 1.11. Give the opposites of the following words taken from text 1.1 and using them make up sentences of your own.

Often, large, allow, generally, separate from, integration, familiar, relative, ease, transmission, primary, simple, constantly, same, true, tiny, fully, arbitrary, fast, previous.

Exercise 1.12. Find in text 1.1 the equivalents for:

form (constitute), cover, use, with the help of (using, with the use of), comprise, allied, evaluation (assessment, appraisal), from 0 to 9, work, usually (commonly, typically), implement, method, coincide partly, regard as (deem, view as), main (principal), fundamentals (principles), suitable, number system, one by one, progress (achievements), restrict, that is to say, give (produce), together with, important, independent of, chain.

Exercise 1.13. Translate the sentences into English using as many equivalents of the italicized words as you can.

Model: Булеву алгебру можна *розглядати* як інструмент, який дозволяє спростити вирази логіки.
– Boolean algebra may be *considered as (regarded as, deemed, viewed as)* a tool that makes it possible to reduce logical expressions.

1. Периферійні пристрої **включають в себе** принтери, модеми, звукові карти та ін. 2. Схема "АБО" **-дає** на виході одиницю, якщо одиниця є хоча би на одному її вході. 3. Алгоритм – це **послідовність** операцій, які треба виконати **одна за одною** для розв'язання даної задачі. 4. У теорії нечітких множин (fuzzy-set theory) функції приналежності (membership functions) мають вигляд **перетинних** площин. 5. Теоретична складова комп'ютерних наук **включає в себе** обчислювальні **методи** і числовий аналіз з одного боку, і структури даних та алгоритми – з іншого. 6. Ці дві теорії, які **частково збігаються**, застосовуються в сучасній обчислювальній техніці та деяких інших **споріднених** галузях знань. 7. Сучасні комп'ютерні науки **охоплюють** надзвичайно широке коло питань: від **основ** теорії обчислень до побудови найскладніших систем штучного інтелекту. 8. Проектування, створення та випробування нової системи становить **основну** частину конструкторської **роботи**. 9. Галузі, що їх **розглядають** як високотехнологічні, **включають в себе** інформаційні технології, біотехнології, нанотехнології та молекулярні нанотехнології. 10. Застосування даної системи **обмежується** низькою продуктивністю деяких функціональних блоків, які досі в ній **застосовуються**. 11. Людський мозок

складається з мільярдів нейронів, які **утворюють** фізичну систему для мислення. 12. Шістнадцяткова **система числення застосовує** цифри **від 0 до 9** та букви **від A до F**. 13. Завдяки **успіхам** у розвитку комп'ютерних наук виникла низка дисциплін, **а саме**: інженерія розроблення програмного забезпечення, комп'ютерна безпека, мультимедійні засоби, які нині вважаються **незалежними** одна від одної. 14. У сучасному електронному обладнанні **важливим**, з-поміж іншого, є розмір, ціна та споживна потужність, тому велика увага приділяється нині спрощенню схем **поряд зі** зменшенням їхніх розмірів. 15. Завдяки малому розміру, високій надійності та великому об'єму пам'яті сучасні флешки стали **зручним** і, як наслідок, дуже поширеним пристроєм зберігання даних.

Exercise 1.14. *Speak on the following issues.*

1. Computer science and its major subdisciplines.
2. The roots of computer science.
3. The merits of George Boole, Alan Turing, and John von Neumann.

Exercise 1.15. *Study the vocabulary to text 1.2.*

application – прикладна програма

assembly language – мова асемблера

mnemonics – мнемоніка

encode – кодувати

linking loader – зв'язуючий завантажувач (завантажувач із редагуванням зв'язків)

build up – нарощувати

increasingly – дедалі більше (й більше)

notion – ідея, поняття

reuse – багатократне використання

sufficiently – досить

compiler – компілятор

efficient – ефективний

consumption – споживання

impetus – поштовх, стимул, імпульс

system-resident software – резидентне ПЗ

handle – керувати, оперувати

trace – простежити, встановити

devise – розробляти

pursuit – прагнення, домагання; гонитва

insert – вставляти

delete – видаляти; знищувати

locate – розміщувати

constitute – становити, становити собою

virtually – фактично

notably – особливо

predate – трапитися раніше чого-небудь, випереджати у часі

coin – створювати

display – 1) відображення; 2) дисплей; 3) відображати

crude – сирий, недосконалий

image – зображення

plot – ділянка

cathode-ray tube (CRT) – електронно-променева трубка

representation – представлення

rectangular – прямокутний

array – матриця, сітка

bit-map graphics – растрова графіка

bit-map display – дисплей із поелементним відображенням, растровий дисплей

commonly – зазвичай, як правило

amber – 1) янтар, бурштин; 2) бурштиновий

high resolution – висока роздільна здатність

explosive – вибуховий, стрімкий

arise – поставати, виникати

attempt – попытка

Exercise 1.16. *Learn the following speech patterns.*

A

Never before had I seen such an excellent picture.

Hardly had I left the room when the telephone rang.

B

His behaviour **is not capable of** explanation.

This device **is capable of** giving temperature indication.

C

She **did not phone me until last week**.

It was not until yesterday that she learnt this news.

D

Thick fog **kept the planes from taking off**.

What **kept you from making** a decision?

Exercise 1.17. *Read and translate text 1.2.*

TEXT 1.2. COMPUTER SCIENCE: THE HISTORY OF DEVELOPMENT

The needs of users and their applications provided the main driving force in the early days of computer science, as they still do to a great extent today. The difficulty of writing programs in the machine language of 0s and 1s led first to the development of assembly language, which allows programmers to use mnemonics for instructions (e.g., ADD) and symbols for variables. Such programs are then translated by a program known as an assembler into the binary encoding used by the computer. Other pieces of system software known as linking loaders combine pieces of assembled code and load them into the machine's main memory unit, where they are then ready for execution. The concept of linking separate pieces of code was important, since it allowed libraries of programs to be built up to carry out common tasks – a first step toward the increasingly emphasized notion of software reuse. Assembly language was found to be sufficiently inconvenient that higher-level languages (closer to natural languages) were invented in the 1950s for easier, faster programming; along with them came the need for compilers, programs that translate high-level language programs into machine code. As programming languages became more

powerful and abstract, building efficient compilers that create high-quality code in terms of execution speed and storage consumption became an interesting computer science problem in itself.

Increasing use of computers in the early 1960s provided the impetus for the development of operating systems, which consist of system-resident software that automatically handles input and output and the execution of jobs. Throughout the history of computers, the machines have been utilized in two major applications: 1) computational support of scientific and engineering disciplines and 2) data processing for business needs. The demand for better computational techniques led to a re-surgence of interest in numerical methods and their analysis, an area of mathematics that can be traced to the methods devised several centuries ago by physicists for the hand computations they made to validate their theories. Improved methods of computation had the obvious potential to revolutionize how business is conducted, and in pursuit of these business applications new information systems were developed in the 1950s that consisted of files of records stored on magnetic tape. The invention of magnetic-disk storage, which allows rapid access to an arbitrary record on the disk, led not only to more cleverly designed file systems but also, in the 1960s and 70s, to the concept of the database and the development of the sophisticated database management systems now commonly in use. Data structures, and the development of optimal algorithms for inserting, deleting, and locating data, have constituted major areas of theoretical computer science since its beginnings because of the heavy use of such structures by virtually all computer software – notably compilers, operating systems, and file systems. Another goal of computer science is the creation of machines capable of carrying out tasks that are typically thought of as requiring human intelligence. Artificial intelligence, as this goal is known, actually predates the first electronic computers in the 1940s, although the term was not coined until 1956.

Computer graphics was introduced in the early 1950s with the display of data or crude images on paper plots and cathode-ray tube (CRT) screens. Expensive hardware and the limited availability of software kept the field from growing until the early 1980s, when the computer memory required for bit-map graphics became affordable. A bit map is a binary representation in main memory of the rectangular array of points (pixels, or picture elements) on the screen. Because the first bit-map displays used one binary bit per pixel, they were capable of displaying only one of two colours, commonly black and green or black and amber. Later computers, with more memory, assigned more binary bits per pixel to obtain more colours. Bit-map technology, together with high-resolution display screens and the development of graphics standards that make software less machine-dependent, has led to the explosive growth of the field. Software engineering arose as a distinct area of study in the late 1970s as part of an attempt to introduce discipline and structure into the software design and development process.

Exercise 1.18. *Find in text 1.2 the English for:*

мова асемблера; магнітна стрічка; цікавий сам по собі; протягом усієї історії комп'ютерів; великою мірою; попит на кращі обчислювальні методи; код, що його транслювали з мови асемблера; від-родження інтересу до числових методів; системне програмне забезпечення; двійкове кодування; резидентне програмне забезпечення; окрема сфера досліджень; потреба в компіляторах; обробка даних; файли записів; растрової графіки; складні системи керування базами даних; на ранній стадії розвитку комп'ютерних наук; підтверджувати теорію; інтенсивне використання; широко застосовуватися; дисплей із високою роздільною здатністю; зв'язуючий завантажувач; машинна мова; мова високого рівня; з точки зору швидкості виконання; вважатися такими, що; основна рушійна сила; технічні дисципліни.

Exercise 1.19. *Translate into English paying special attention to the italicized words.*

1. Досягнення в галузі комп'ютерних наук **значною мірою** залежать від рівня розвитку фундаментальних наук. 2. Операційна система складається з **резидентного програмного забезпечення**, яке автоматично керує введенням, виведенням та виконанням завдань. 3. **Системне програмне забезпечення** зазнавало постійних змін **протягом всієї історії комп'ютерів**. 4. З одного боку, **мови високого рівня** є зручними для людини, але з іншого, **мови низького рівня**, як, наприклад, **машинна мова** або **мова асемблера**, є зручнішими для машини. 5. Розробка **складних систем керування базами даних вважається окремою сферою досліджень**. 6. Розробка моніторів, які мали би **дисплей із високою роздільною здатністю, вважається** важливим завданням обчислювальної техніки. 7. Булева алгебра, що була створена **на ранній стадії розвитку комп'ютерних наук**, описує операції над числами в двійковій системі числення. 8. **Обчислювальні методи**, що застосовуються для **обробки даних** у цифрових комп'ютерах, базуються на законах булевої алгебри. 9. **Інтенсивне використання** компакт-дисків у сучасних комп'ютерах пояснюється їхнім великим об'ємом пам'яті. 10. Зростаюча потреба у різному прикладному та **системному програмному забезпеченні** стала **основною рушійною силою** розвитку такої **технічної дисципліни**, як інженерія розробки програмного забезпечення.

Exercise 1.20. *Say whether the following sentences are true or false. Correct the false ones.*

1. The difficulty of writing programs in the machine language of 0s and 1s led first to the development of assembly language, which allows programmers to use mnemonics for variables and symbols for instructions. 2. Other pieces of application software known as language processors combine pieces of assembled code and load them into the machine's main memory unit. 3. The concept of linking separate pieces of code was important, since it allowed libraries of programs to be built up to carry out common tasks. 4. Increasing use of computers in the early 1960s provided the impetus for the development of operating systems, which consist of system-resident software that automatically translates high-level language programs into machine code. 5. Numerical methods and their analysis is an area of physics that can be traced to the methods devised several centuries ago by physicists for the hand computations they made to apply their theories. 6. The invention of magnetic-disk storage allowed rapid access to an arbitrary record on the disk. 7. Computer graphics was introduced in the early 1950s with the display of data or perfect images on cathode-ray tube screens and liquid crystal displays. 8. A bit map is a binary representation in main memory of the rectangular array of points (pixels, or picture elements) on the screen.

Exercise 1.21. *Complete the sentences translating their Ukrainian parts into English.*

1. (Мова асемблера виявилася досить незручною, тому) high-level languages (closer to natural languages) were invented in the 1950s for easier, faster programming. 2. As programming languages became more powerful and abstract, (побудова ефективних компіляторів, які створюють високоякісний код із точки зору швидкості виконання та споживання пам'яті, стало цікавим завданням обчислювальної техніки). 3. (Протягом усієї історії комп'ютерів) the machines have been utilized in two major applications: (обчислювальне забезпечення наукових і технічних дисциплін) and data processing for business needs. 4. Improved methods of computation had the obvious potential (зробити революційні зміни у веденні справ). 5. The invention of magnetic-disk storage, which allows rapid access to an arbitrary record on the disk, led not only to more cleverly designed

file systems but also, in the 1960s and 70s, (до поняття бази даних і розробки складних систем керування базами даних, які широко застосовуються нині). 6. Data structures, and the development of optimal algorithms for inserting, deleting, and locating data, have constituted major areas of theoretical computer science since its beginnings (через інтенсивне використання таких структур практично всім програмним забезпеченням комп'ютера). 7. Artificial intelligence actually predates the first electronic computers in the 1940s, (хоча сам цей термін було винайдено лише у 1956 році). 8. Bit-map technology, together with high-resolution display screens (і розробкою стандартів графіки, які роблять програмне забезпечення менш машиннозалежним, привели до стрімкого зростання цієї галузі).

Exercise 1.22. *Translate into English.*

1. Потреби користувачів та їхніх прикладних задач стали головною рушійною силою на ранній стадії розвитку комп'ютерних наук. 2. Потім такі програми транслюються програмою, відомою як асемблер, у двійковий код, який використовується комп'ютером. 3. Поряд із мовами високого рівня з'явилася потреба у компіляторах – програмах, які транслюють програми на мовах високого рівня у машинний код. 4. Попит на кращі обчислювальні методи привів до відродження інтересу до числових методів та їх аналізу. 5. У 1950-х роках були розроблені нові інформаційні системи, які складалися з файлів записів, що зберігалися на магнітній стрічці. 6. Ще однією метою комп'ютерних наук є створення машин, здатних виконувати завдання, які вважаються такими, що потребують людського інтелекту. 7. Висока вартість апаратного забезпечення та нестача програмного забезпечення стримували розвиток цієї галузі до початку 1980-х років, коли пам'ять, необхідна для растрової графіки, стала доступною. 8. Оскільки перші растрові дисплеї використовували один двійковий розряд на піксель, вони були здатні відображати лише один із двох кольорів. 9. Пізніші комп'ютери, що мали більшу пам'ять, відводили більше двійкових розрядів на піксель з тим, щоби отримати більше кольорів. 10. Інженерія розроблення програмного забезпечення постала як окрема сфера досліджень наприкінці 1970-х як спроба упорядкувати і структурувати процес проектування і розробки програмного забезпечення.

Exercise 1.23. *Answer the questions.*

1. What caused the appearance of assembly language? In what way does it differ from machine language? 2. What is the difference between assembler language and high-level languages? 3. What is the function of a linking loader? 4. What are the assembler and compiler designed for? 5. What is the operating system? 6. In what major applications have computers been utilized throughout all their history? 7. Where were data stored in information systems developed in the 1950s? 8. What did the invention of magnetic-disc storage provide? 9. What machines are called intellectual? 10. What restrained the development of computer graphics until the 1980s? 11. What is bit-map graphics? 12. Why were the first bit-map displays two-coloured? In what way were a greater number of colours attained?

Exercise 1.24. *Give derivatives of the following words and explain their meanings.*

Need, force, extent, symbol, assemble, translate, program, code, load, memory, separate, invent, speed, create, require, colour, distinct.

Exercise 1.25. *Give the opposites of the following words taken from text 1.2 and using them make up sentences of your own.*

Main, early, difficulty, separate (*adj.*), important, common, powerful, abstract, create, load, interesting, automatically, major, better, obvious, cleverly, theory, crude, growth, explosive, introduce.

Exercise 1.26. *Speak on the following issues*

1. The development of computer languages.
2. The development of computers since the 1960s.
3. Computer graphics.

Grammar: Tenses of the Verb.

Sequence of Tenses

Exercise 1.27. *Use the proper tense form of the verbs in brackets.*

1. The students (go) to the party yesterday. 2. We (look) at the calculating device made in the XVII century. 3. They (change) their teachers every year. 4. They (interrupt) our talk. 5. My friend (work) at this project three months ago. 6. Our lecturer (be) at the lecture at 8 o'clock. 7. She (stay) at home tomorrow. 8. I (ask) him about it before he (go). 9. I (wait) for my friend when the phone (ring). 10. I'll come at 3 o'clock. – Good. I (wait) for you. 11. If I receive any message from him I (let) you know. 12. He (know) me for over 10 years. 13. We lived here when I (be) five. 14. I just (tell) you the answer. 15. My friends (go) away five minutes ago. 16. My roommate wasn't at home when I arrived. He just (go) out. 17. After they (show) their passports to the officials at the airport they (get) on the bus and (go) to the Terminal. 18. When my thoughts are written down I (finish) with them and I shall have time to think of other things. 19. I suppose by then they (settle) all the problems. 20. We (study) at this university for two years. 21. We (wait) for several days before he (answer) our questions. 22. My friend (tell) me yesterday he (be) very happy that he (graduate) from the university. 23. Don't (go) to Peter's place right now, he (be) busy. He (write) a new program for his project. If you (come) after six he (be) very glad and (demonstrate) it to you. 24. If you (help) me I (do) this work well. 25. There (be) the time when only privileged people (have) an opportunity to learn the basics, called the three R's: reading, writing, and arithmetics. 26. Computers (become) commonplace in homes, offices, stores, schools, research institutes, plants; the use of computers in business, industry and communication services (be) widespread today. 27. The system did not work because they (connect) the loudspeaker wrongly. 28. While they (carry out) tests in the laboratories, researches (analyze) past results. 29. Many people today (telecommute) – that is, (use) their computers to stay in touch with the office while they (work) at home. 30. Helen (learn) English at the university. She (learn) English since last autumn.

Exercise 1.28. *Choose the right form of the verb in brackets. Mind the sequence of tenses.*

1. I said, "I ... you, and ... probably persuade you to come" (know/knows/knew; can/could/will be able to). 2. The dean asked them several times what ... (happens/happen/is happening/has happened/had happened). 3. I asked her what her name

... and she said "Try to guess!" (am/is/are/was/were). 4. My friend dragged me here. He said it ... good for us to get these lectures (is/are/was/were). 5. He asked her if she ... to be in London for long (is going/was going/were going). 6. I didn't know you ... here (is/was/are/were). 7. Our boss expected that his subordinate ... the first opportunity to apologize for his rudeness (take/takes/will take/would take). 8. They ... yesterday that the IT company ... all accounts promptly (are informed/have informed/ were informed; paid/ have paid/ has paid). 9. He asked me where I ... (study/ studied/ was studying). 10. We ... that many system administrators and programmers ... for new inventions, designs and production processes (find out/ found out/ have found out; have been awarded/ were awarded/ had been awarded).

Exercise 1.29. Change the sentences into indirect speech.

1. The professor told his student, "You've written an excellent program". 2. The salesman said to us, "This is the best version of a general-purpose computer". 3. My friend told me, "We have plenty of time to do our work". 4. The lecturer said to me, "You will make a report on computer science". 5. "Did they understand what you said to them?" he asked. 6. My friend said to me, "Collect all the needed devices for our laboratory work". 7. "We have a computer but very often it doesn't work", they said. 8. The teacher came to the class-room and asked the students, "What are you doing?" 9. The professor told me, "Be ready to take part in the conference". 10. I asked my group-mate, "Are you going to repair you computer on Saturday or on Sunday?"

Exercise 1.30. Translate into English. Mind the sequence of tenses.

1. Він сказав, що дискусія триває. 2. Вона повідомила про те, що бачила. 3. Я сумнівався, чи він був правий. 4. Наступного разу я зроблю, як він говорить. 5. Я думав, що ти зайнятий. 6. Ми знаємо, що ти зайнятий. 7. Він запитав мене, де я купив цей підручник. 8. Ми не знали, що вона хвора. 9. Завтра я запитаю його, чого він хоче. 10. Цікаво, чому випуск нової версії цього програмного забезпечення затримується. 11. Він помітив, що монітор не працює. 12. Вона зауважила, що стає пізно. 13. Я обіцяв, що прийду. 14. Кількість обрахунків вказувала, що це має бути програма значного об'єму. 15. Він сказав, що знає німецьку та французьку мови. 16. Я була впевнена, що староста групи повідомив вас про те, що наступного разу ми відвідаємо виставку новітніх технологій. 17. Я була впевнена, що вони будуть раді бачити нас. 18. Я думав, що він – викладач. 19. Ми бачили, що викладач задоволений результатами нашого іспиту. 20. Ми були впевнені, що наша доповідь про розвиток комп'ютерної науки отримає перший приз.

Exercise 1.35. Study the vocabulary to text 1.4.

originally – спочатку

origin – походження

count – лічити

extensively – широко

various – різноманітний

human activities – людська діяльність

diverse – різний, різноманітний

accountant – бухгалтер

researcher – дослідник

investigation – дослідження

store – зберігати

volume – обсяг

computer-aided, computer-assisted – автоматизований

computer-aided design – автоматизоване проектування

industrial process control – керування технологічним процесом

lastly – зрештою

captivating – захоплюючий

retrieve – здійснювати пошук (*інформації*)

process – обробляти

processing – обробка

word processing – обробка текстів

device – пристрій

operation – 1) робота, функціонування (*технічного пристрою*); 2) операція

specially arranged – спеціально, в особливий спосіб організований

machinery – 1) устаткування; 2) корпус

special-purpose computer – комп'ютер спеціального призначення

general purpose computer – комп'ютер загального призначення

embed – вмонтовувати

appliance – пристрій

wristwatch – наручний годинник

preprogram – запрограмувати наперед

tune – настроювати

personal digital assistant (PDA) – електронний секретар

notepad – блокнот

scheduling system – планувальник

cellular phone – стільниковий телефон

computer network – комп'ютерна мережа

regardless of – незважаючи на, незалежно від, попри

laptop computer – портативний комп'ютер

versatile – універсальний

enable – давати можливість, дозволяти

track finances – вести облік фінансів

keyboard – клавіатура

trackball – трекбол (кульковий маніпулятор)

pointing device – пристрій управління курсором ("миша", світлове перо, джойстик)

video display monitor – відеомонітор

liquid crystal display (LCD) – рідкокристалічний дисплей

manipulating capabilities – можливості обробки (*інформації*)

handle – оперувати, маніпулювати

records – документація

accounting records – бухгалтерська документація

inventory records – інвентарна документація

workstation – автоматизоване робоче місце (АРМ)

exchange – обмінюватися

mainframe computer – мейнфрейм, комп'ютер великої потужності (*як правило, використовується в режимі розподілу часу, для наукових розрахунків, для керування розподіленою системою*)

speed – швидкість

attain – досягати

Exercise 1.36. Read and translate text 1.4.

TEXT 1.4. MODERN COMPUTERS

Originally computers were meant to perform mathematical calculations. This accounts for the origin of the word itself coming from the Latin word *computo* which means *to count*. The development of electronic computation, however, has resulted in the fact that modern computers are used extensively in various fields of human activities, not for calculations only; and these fields are becoming more and more diverse. Now you can hardly imagine an accountant's desk without a computer, to say nothing of a research laboratory. Modern computers can solve a great variety of tasks: to perform mathematical calculations, to help researchers in their investigations, to help doctors in making diagnoses, to store large volumes of information, etc. And who has not heard about computer-aided design, or computer-assisted industrial process control and fault diagnosis, or computer-based learning, or, lastly, computer games which are extremely captivating not for children only but for adults as well? Thus, at its simplest, the computer can be defined as a programmable electronic device that can store, retrieve and process information (data). The word *programmable* here means that computer's operation is based on a program – a specially arranged list of instructions. Every computer incorporates hardware and software. The former includes machinery and devices, the latter – all computer systems and programs. According to the purpose the following classification of computers may be suggested.

Special-purpose, or dedicated, computers. They are designed to perform special tasks; their operations are limited to the programs built into their microchips. The smallest are embedded within the circuitry of appliances, such as TV-sets and wristwatches. These computers are preprogrammed for a specific task, such as tuning to a particular television frequency or keeping accurate time. Among special-purpose computers there are also so called personal digital assistants (PDAs). These computers can be held in one hand and are used as notepads, scheduling systems, and address books; if equipped with a cellular phone, they can connect to worldwide computer networks to exchange information regardless of location.

General-purpose computers, such as personal computers (PC) and laptop computers. They are much more versatile because they can accept new sets of instructions. Each new set of instructions (program) enables the same computer to perform a different type of operation. Laptop computers and PCs are typically used in businesses and at home for word processing, to track finances, to play games, and to communicate on computer networks. They have large amounts of internal memory to store hundreds of programs and documents. They are equipped with a keyboard, a mouse, a trackball or other pointing devices and a video display monitor or liquid crystal display (LCD) to display information. Laptop computers usually have similar hardware and software as PCs, but they are more compact and have flat, lightweight LCDs instead of video display monitors.

Minicomputers are fast computers that have greater manipulating capabilities than personal computers and can be used simultaneously by many people. These machines are primarily used by larger businesses to handle extensive accounting and inventory records.

Workstations are similar to personal computers but have greater memory and more extensive mathematical abilities, and they are connected to other workstations or PCs to exchange data. They are typically found in scientific, industrial, and business environments that require high levels of computational abilities.

Mainframe computers have more memory, speed, and capabilities than workstations and are usually shared by multiple users through a series of interconnected computers. They are large, extremely fast, multi-user computers that often contain complex arrays of processors, each designed to perform a specific function. Mainframes control businesses and industrial facilities and are used for scientific research.

The most powerful mainframe computers also called *supercomputers* are the fastest class of computers. Their speed and power are almost beyond human comprehension. Supercomputers process complex and time-consuming calculations, such as those used to create weather predictions. They attain these speeds through the use of several advanced engineering techniques. Because these computers can cost millions of dollars, they are used mostly by government, large businesses, scientific institutions and the military.

Exercise 1.37. *Find in text 1.4 the English for:*

виконувати обчислення, дослідницька лабораторія, нарізно-манітніші завдання, встановити діагноз, діагностика несправностей, автоматизоване проектування, програмований пристрій, здійснювати обмін інформацією, внутрішня пам'ять, широкі можливості, одночасно, наукові дослідження, за межами людської уяви, прогноз погоди, передові технології.

Exercise 1.38. *Complete the sentences translating their Ukrainian parts into English.*

1. (Навряд чи можна уявити) an accountant's desk without a computer, (не кажучи вже про дослідницьку лабораторію). 2. Every computer (об'єднує апаратне та програмне забезпечення). 3. Special-purpose computers (призначені для виконання конкретних задач). 4. The program is (організований в особливий спосіб список команд). 5. Personal computers are typically found in businesses and at home (для редагування текстів, ведення фінансового обліку, ігор та спілкування в комп'ютерних мережах). 6. Laptop computers have large amounts of (внутрішньої пам'яті для зберігання сотень програм і документів). 7. Minicomputers have greater (можливості роботи з інформацією, ніж персональні комп'ютери). 8. Workstations are connected to other workstations or personal computers (для обміну інформацією). 9. Mainframes control businesses and industrial facilities and are used (для наукових досліджень). 10. Supercomputers process complex and time-consuming operations, (такі, які використовуються для створення прогнозів погоди).

Exercise 1.39. *Translate into English.*

1. Від самого початку комп'ютери призначалися для виконання математичних обчислень. 2. Комп'ютерні ігри є надзвичайно захоплюючими не лише для дітей, а й для дорослих. 3. Апаратне забезпечення включає в себе устаткування та пристрої, а програмне забезпечення – всі комп'ютерні системи та програми. 4. Функції комп'ютерів спеціального призначення обмежуються програмами, вмонтованими в їхні мікросхеми. 5. Кожна нова програма дає можливість одному й тому самому комп'ютеру виконувати інші операції. 6. Мінікомп'ютери можуть використовуватися одночасно багатьма людьми. 7. Автоматизовані робочі місця подібні до персональних комп'ютерів, але мають більший об'єм пам'яті. 8. Швидкодія та потужність суперкомп'ютерів є майже недоступними людській уяві. 9. Комп'ютер – це програмований електронний пристрій, що може зберігати, здійснювати пошук та оброблення інформації. 10. Найменші комп'ютери вмонтовують у схеми пристрою, такого, як, наприклад, годинник або телевізор. 11. Комп'ютери мають можливість виконувати декілька завдань одночасно, витрачаючи мінімум часу та зусиль.

Exercise 1.40. *Answer the questions.*

1. What was the original purpose of computers? 2. What has the development of electronic computation resulted in? 3. What kind of tasks can modern computers solve? 4. How can the computer be defined? 5. What does the word *programmable* mean? 6. What are dedicated computers designed for? 7. How are personal digital assistants usually used? 8. Why are general-purpose computers versatile? 9. Where are laptop computers and personal computers typically used? 10. What are the advantages of computers? 11. Where are workstations typically found? 12. What is the fastest class of computers? 13. Owing to what

do supercomputers attain their high speeds? 14. What accounts for the limited applications of supercomputers? 15. Why do we say that computers made our life easier?

Exercise 1.41. *Speak on the following.*

1. Modern computer and its applications.
2. Special-purpose and general-purpose computers.
3. Classification of computers according to their computational abilities.

Exercise 1.42. *Memorize the vocabulary below, then read and translate text 1.5.*

Logical relation – логічне відношення

computing – обчислення

high – level programming language – мова програмування високого рівня

genius – геній

recognition – визнання

deserve – заслуговувати

conceive – вважати

draw up – складати

devise – придумувати, винаходити; розробляти

contribute – сприяти

boot up – завантажувати

TEXT 1.5. FAMOUS PEOPLE IN THE HISTORY OF COMPUTER TECHNOLOGY

Almost everyone uses computers these days for everything from shopping to working. But have you ever stopped to think about where all this amazing technology came from? Who invented it all? Well, behind every company, programming language or piece of software, there is a person - or sometimes a team of people - who turned ideas into reality. We've all heard of Bill Gates, the founder of Microsoft and one of the richest men in history. Equally famous is Steve Jobs, the person who, along with Steve Wozniak, started Apple computers. However, there are hundreds of other people, from early pioneers to later geniuses, who aren't as well known but who deserve recognition for the work they did in advancing the world of computing.

One of the first people to conceive of computers was Charles Babbage, an English mathematician and analytical philosopher who drew up plans for the first programmable computer called the Difference Engine. George Boole came up with a way of describing logical relations using mathematical symbols - now called Boolean logic - that is the basis of

all modern computer processes. Another notable figure in early computing was Alan Mathison Turing, an Englishman known as the "father of computer science". He invented the Turing Test, which is a way to find out if a computer is acting like a machine or a human.

As computing became more complicated, people needed a way to make it easier to program the computers. These computer instruction systems became known as computer, or programming, languages. FORTRAN, the first widely used high-level programming language was invented by an American computer scientist, John Warner Backus. Other notable North American inventors of programming languages include Dennis Ritchie, author of the C programming language, Larry Wall, creator of Perl, and Canadian James Gosling, known as the father of Java. Two men from Denmark are responsible for writing two other famous programming languages. Bjarne Stroustrup came up with C++ and Rasmus Lerdorf devised PHP. Dutchman Guido van Rossum wrote the Python programming language, while the Japanese computer scientist, Yukihiro Matsumoto, made a language called Ruby. One of the uses of programming languages is to create operating systems, which are essentially sets of instructions that allow computers to function.

The most widely-used operating system in the world is Microsoft Windows, but there are other powerful ones that exist, such as Unix, created by Ken Thompson and his team at AT&T in 1969, and Linux, written by Linus Torvalds in 1991. Microsoft, of course, is the largest software company in the world, but there is another company, Intel, that is equally important when it comes to hardware. Intel was started by several people who are now legends in the computer world, including Robert Noyce and Gordon Moore.

Through their creativity and hard work, all of these people contributed to shaping what we now experience as Information and Computer Technology. Every time you boot up a computer, play a video game or surf the Internet, try to remember the individuals who made these wonders possible.

Exercise 1.43. Give *Ukrainian equivalents of the following word combinations*.

From early pioneers to later geniuses; the advancing world of computing; analytical philosopher; another notable figure; one of the uses of programming languages; to be started by several people; to experience as Information and Computer Technology; everything from shopping to working; behind every company; along with; to become known as computer or programming languages.

Exercise 1.44. *Find in text 5 the English for:*

дивовижні технології; втілювати ідеї в життя; складати план; з використанням математичних символів; спосіб пізнання; створювати операційні системи; бути однаково важливим; найбільша компанія з розробки програмного забезпечення; завдяки творчості і напруженій роботі; зробили диво реальністю; так само відомий; заслуговувати визнання; робити успіхи в обчислювальній галузі; спосіб описання логічних відношень; основа всіх сучасних обчислень; полегшувати програмування комп'ютерів; легенди комп'ютерного світу.

Exercise 1.45. *Answer the questions on text 1.5.*

1. What does everyone use computers for these days? 2. What is behind every company, programming language or piece of software? 3. Who started Apple computers? 4. Who was one of the first people to conceive of computers? 5. What did George Boole come up with? 6. Who is another notable figure in early computing? 7. What did people need as computing became more complicated? 8. What was the first widely used high-level programming language? 9. Who is the author of the C programming language? 10. What is the most widely-used operating system? 11. Whom was Unix created by? 12. What are well-known software companies? 13. Who was Intel started by? 14. What should people do every time they boot up a computer?

Exercise 1.46. *Comment on the role of famous people in the history of computer technology.*

Grammar: Passive Voice

Exercise 1.47. *Change the sentences into the Passive Voice.*

1. Modern computers can solve a great variety of tasks. 2. The specialists introduced computer graphics in the 1950s. 3. Mainframes control businesses and industrial facilities. 4. Supercomputers process complex and time-consuming calculations, such as those used to create weather predictions. 5. Electronic devices are doing simple but human-like thinking. 6. Computers use a form of digital information called binary information. 7. Analog computers perform mathematical operations on continuous electric values. 8. Analog refers to numerical values that have a continuous range. 9. The demand for better computational techniques caused a re-surgence of interest in numerical methods and their analysis. 10. A computer can take in information, perform different operations and provide answers. 11. Electronic devices have revolutionized our life. 12. We may consider a hybrid computer as a combination of digital and analog ones. 13. We use communication devices to send information to or from external storage. 14. Today computers have profoundly changed the way in which people do many kinds of work. 15. The input devices receive signals from the control unit.

Exercise 1.48. *Change the sentences into the Active Voice.*

1. Personal digital assistants can be held in one hand and are used as notepads, scheduling systems, and address books. 2. Because supercomputers can cost millions of dollars, they are used mostly by government, large businesses, scientific institutions and the military. 3. In the binary number system all information can be represented as sequences of zeros and ones. 4. The telephone and radio were constructed using the principles of analog electronics. 5. The computer will have been repaired before you come. 6. Such programs are then translated by a program known as an assembler into the binary encoding used by the computer. 7. The field was kept from growing until the early 1980s by expensive hardware and the limited capability of software. 8. The 0.95 will be interpreted by the computer as a 1, and no data will be lost as a result. 9. Minicomputers are primarily used by large businesses to handle extensive accounting. 10. Digital computers can be programmed by programmers to perform a host of varied tasks. 11. In recent years, hybrid computers have been used by scientists in simulation studies of spacecraft. 12. Computer operating system is often stored in a computer's ROM memory. 13. A great step forward in the art of mechanical adding was made by Blaise Pascal in 1642. 14. John von Neumann's main idea was that all instructions as well as data should be stored in the computer's memory for

fast access and execution. 15. After processing information is usually printed out on a printer.

Exercise 1.49. Ask **when-** and **where-**questions using the *Passive Voice*.

Model: He bought a computer two years ago. – When was a computer bought by him?

1. I always check programs before making beta testing. 2. This company will produce a new model of a digital computer next year. 3. The programmer stored all the results of processing in the computer's memory. 4. We met our competitor at the IT exhibition. 5. They have made millions of computers at this plant since that time. 6. Our group will write a qualification test tomorrow. 7. They spoke about new calculating devices at the conference. 8. Logical elements can take on many forms in the CPU. 9. They developed the magnetic core, the transistor and many other solid state devices thirty years ago. 10. The machine stores all the instructions in the memory locations. 11. When only one address is specified, the computer executes the given command upon the operand found in the location specified by the address. 12. Programmers usually prepare a complete program of instruction words and a list of required data words beforehand. 13. The computer's input device (a disk drive or tape drive) reads the information into the computer.

Exercise 1.50. *Translate into English using the Passive Voice.*

1. У сфері програмного забезпечення було зроблено багато вдосконалень. 2. Ці програми призначені для використання спеціалістами у сфері інформаційних технологій та звичайними користувачами. 3. Перед записом на диск усі фото було відскановано. 4. До появи комп'ютерів люди виконували безліч складних завдань. 5. Комп'ютерну графіку використовують для виконання складних та специфічних завдань на виробництві. 6. Програму, написано мовою високого рівня, необхідно транслювати на машинну мову. 7. Комп'ютер можна визначити як пристрій, який обробляє інформацію відповідно до набору команд, що називається програмою. 8. Транзистор винайшли американські науковці у 1948 році. 9. Нові типи інтегральних схем збільшили щільність запису. 10. Науковці розробляють нові способи вдосконалення інтегральних схем.

Unit 2. Computer Software

Exercise 2.1. *Study the vocabulary to text 2.1.*

control – керувати

specify – задавати, визначати

meaningful – значущий

ever-larger – постійно зростаючий

arise (arose, arisen) – з'явитися, постати

perceive – 1) розуміти, усвідомлювати; 2) сприймати

care – ретельність, уважність, обережність

maintainability – експлуатаційна надійність; зручність експлуатації

application software – прикладне програмне забезпечення

system software – системне програмне забезпечення

one-off – одноразовий, (*тут* спеціальний)

alternatively – або

software house – фірма з розробки та реалізації програмних засобів

payroll – платіжна відомість (*тут* назва відповідної прикладної програми)

pay slip – бланк платіжної відомості

word processing – оброблення (редагування) тексту

ultimately – зрештою, в кінцевому підсумку

machine language – машинна мова

assembly language – мова асемблера

closeness – близькість, наближеність

translator program – програма-транслятор

run – виконувати (*програму*)

debug – налагоджувати (*програму*)

opcode – код операції

language processor – транслятор або інтерпретатор (програма, яка транслює або інтерпретує програму, написану мовою програмування)

handle – оперувати, працювати з чимось

therefore – тому, внаслідок чого

one-to-one – однозначний

trend – тенденція

remote – 1) віддалений; 2) несхожий

Exercise 2.2. *Learn the following speech patterns.*

A

We did all our best for **successfully completing** the job.

I hate your **constantly interrupting** me.

B

Electrons can **be made to travel** at very high speeds.

We **were made to learn** fifty new words every week.

C

It's disgraceful **the way he behaves**.

The work must be finished one **way** or another.

D

A construction which may **substitute for** a word is a phrase.

The actress's big break came when she **substituted for** the ailing star.

E

Assembly language is **harder to use** than a high-level language.

COBOL is **convenient to use** for simple business applications.

F

Electric current will **either** flow through a transistor **or** not flow depending on the base voltage.

Energy can be **neither** created **nor** destroyed.

G

A cone is **much like** a pyramid but has a circle for a base.

A modern PC is **nothing like** a bulky computer used in the 1960s-1970s.

Exercise 2.3. Read and translate text 2.1.

TEXT 2.1. APPLICATION SOFTWARE. COMPUTER LANGUAGES

A digital computer cannot work without software, i.e. a program which controls different hardware components of the computer in a sequence specified by the user. By simply changing the program, the computer can be made to work in a different manner. In other words, software provides a digital computer with general-purpose computing capability. One should keep in mind that software and hardware are complementary to each other. Both have to work together to produce a meaningful result. The software, that is becoming an ever-larger part of the computer system, is growing more and more complicated, requiring teams of programmers and years of effort to develop. As a consequence, a new subdiscipline, software engineering, has arisen. The development of a large piece of software is perceived as an engineering task, to be approached with the same care as the construction of a skyscraper, for example, and with the same attention to cost, reliability, and maintainability of the final product. Computer software is usually classified into two broad categories: application software and system software.

Application software is a set of programs to carry out operations for a specific application. Application software may consist of one-off programs written by the user or alternatively of packages written by commercial software houses and bought by the user to accomplish specific tasks. For example, payroll is an application software for an organization to produce pay slips as an output. Application software is useful for word processing, billing system, accounting, producing statistical reports, analysis of numerous data in research, weather forecasting, etc. Such systems as MS Word, Lotus 1-2-3, dBASE III Plus are all application softwares.

Another example of application software is programming languages. Programming languages are the languages in which a programmer writes the instructions that the computer will ultimately execute. There are two major types of programming languages. These are *low-level languages* and *high-level languages*. Low-level languages are further divided into *Machine language* and *Assembly language*. The term *low-level* means closeness to the way in which the machine has been built. Low-level languages are machine oriented and require extensive knowledge of computer hardware and its configuration.

Machine language is the only language that is directly understood by the computer. It does not need any translator program. Machine-language instructions, which are also called *machine code*, are written in binary form - i.e., strings of 1s and 0s (e.g. 01011010) - because electronic circuits inside computers, including memories, work only with the signals 0 and 1. When this sequence of codes is fed to the computer, it recognizes the codes and converts it into electrical signals needed to run it. For example, a program instruction may look like this: 10110010110100. It is not an easy language for the user to learn. It is also difficult to debug programs written in this language. Thus, programmers use languages that are readable for them but not for computers. For example, the mnemonic opcode LOAD in a human-readable language is far easier to understand than 0101 in machine code. Such a human readable language must be translated into machine language by special purpose programs called language processors. Machine language is considered to be the first generation language.

Assembly language was the first step to improve the programming structure. You know that computer can handle numbers and letters. Therefore some combinations of letters can be used to substitute for numbers of machine codes. The set of symbols and letters forms the assembly language and a translator program is required to translate this language to machine language. This program is called *Assembler*. The symbolic programming of this language is easier to understand and saves a lot of time and effort of the programmer. It is also easier to correct errors and modify program instructions. Assembly language has the same efficiency of execution as the machine level language because there is a one-to-one translator between an assembly language program and its corresponding machine language program. Assembly language is considered to be a second-generation language.

So, you now understand that programming in either assembly language or machine language is not an easy business requiring deep knowledge of computer hardware. To make the programmer's work easier more convenient, high-level languages were soon (beginning in the mid-1950s) invented. High-level languages are problem oriented because the instructions are suitable for solving a particular problem. For example the language FORTRAN (FORMula TRANslation) was invented for solving mathematical problems. It was originally much like assembly language; however it allowed programmers to write algebraic expressions instead of coded instructions for arithmetic operations. COBOL (COMmon Business Oriented Language) was developed to handle records and files and the operations necessary for simple business application. The trend since then has been toward developing increasingly abstract languages, allowing the programmer to think and communicate with the machine at a level ever more remote from machine code.

Exercise 2.4. Find in text 2.1 the English for:

працювати з числами і буквами; обробка текстів; стає дедалі складнішим; по-іншому; як наслідок; конкретна задача; мова низького / високого рівня; програмне та апаратне забезпечення доповнюють одне одне; дедалі більше несхожий на машинний код; у визначеній користувачем послідовності; схожість з тим, як.

Exercise 2.5. Translate into English paying special attention to the italicized words.

1. Персональні комп'ютери та ноутбуки, що зараз вже стали для нас звичними, дуже **несхожі на** обчислювальні машини 60-70-х ро-ків. 2. Цифрові комп'ютери **працюють із числами і буквами**, які представлені двійковим кодом. 3. Гармонійний розвиток особистості досягається поєднанням розумової і фізичної праці, які **доповнюють одна одну**. 4. Мініатюризація елементної бази електроніки призвела до створення інтегрованих мікросхем, і, **як наслідок**, наприкінці минулого століття стали з'являтися комп'ютери, які за своїми розмірами ставали **дедалі меншими**. 5. Для полегшення праці програміста із середини 50-х років розробляються мови, які **дедалі більше схожі** на природну мову. 6. **Схожість із тим, як** було скоєно інший злочин, допомогла поліції швидко встановити особу злочинця. 7. Мови програмування, які схожі на людську мову і **несхожі на** машинний код, і, **як наслідок**, є зручнішими для людини, називаються **мовами високого рівня**. 8. У сучасний футбол, який дуже **не схожий на** той, що був сто років тому, грають зовсім **по-іншому**. 9. Розвиток інформаційних технологій та вдосконалення апаратних і програмних засобів привело до того, що сучасні комп'ютери, з одного боку, **стають дедалі складнішими**, а з іншого - **дедалі зручнішими в користуванні**. 10. Прикладна програма спрямована на розв'язання **конкретної задачі** шляхом виконання операцій **у визначеній алгоритмом послідовності**.

Exercise 2.6. *Say whether the following sentences are true or false. Correct the false ones.*

1. Software provides a digital computer with general-purpose capabilities. 2. Computer software and hardware are interchangeable. 3. As a separate discipline, software engineering appeared in the early days of computer science. 4. Application software may consist of one-off programs written by the user or alternatively of packages written by commercial software houses and bought by the user to accomplish specific tasks. 5. Application software is useful for word processing, billing system, accounting whereas system software is designed for solving research problems. 6. A programming language is a special coding system designed for writing instructions for a computer. 7. Low-level languages are machine-oriented, which means that they are used for solving a narrow circle of problems. 8. Low-level languages require no translator program. 9. High-level languages are used if it is not possible to solve a problem using a low-level language program. 10. High-level languages are termed so because they require high-skilled programmers being very difficult for man to understand and demanding deep knowledge of computer hardware.

Exercise 2.7. *Complete the sentences translating their Ukrainian parts into English.*

1. Software controls the different hardware components of the computer (у визначеній користувачем послідовності). 2. (Іншими словами, програмне забезпечення надає комп'ютеру) general-purpose computing capability. 3. Software and hardware have to work together (щоб дати значущий результат). 4. (Програмне забезпечення, що стає дедалі більшою частиною комп'ютерної системи, стає складнішим і складнішим, потребуючи) teams of programmers and years to develop. 5. (Як наслідок,) a new subdiscipline, software engineering, has arisen. 6. The development of a large piece of software is perceived as an engineering task, (до якої слід підходити з такою самою увагою, як, наприклад, до зведення хмарочоса,) and with the same attention to (вартості, надійності та зручності експлуатації кінцевого продукту). 7. (Прикладне програмне забезпечення може складатися зі спеціальних програм, що їх пише користувач, або з пакетів, які пишуть фірми) and bought by the user to accomplish specific tasks. 8. (Ще одним прикладом прикладного програмного забезпечення) is programming languages. 9. Such a human readable language (треба транслювати в машинну мову за допомогою спеціальної програми, яка називається "транслятор"). 10. The symbolic programming of assembly language (легше для розуміння та економить програмісту багато часу та зусиль). 11. (Аби полегшити програмісту роботу) more convenient, high-level languages were soon invented. 12. (Відтоді існує тенденція, спрямована на розвиток мов дедалі більш абстрактних,) allowing the programmer to think and communicate with the machine (на рівні дедалі більш несхожому на машинний код).

Exercise 2.8. *Translate into English.*

1. Простою зміною програми можна добитися того, що комп'ютер працюватиме по-іншому. 2. Слід пам'ятати, що програмне та апаратне забезпечення доповнюють одне одного. 3. Комп'ютерне програмне забезпечення зазвичай поділяють на два великі класи: прикладне та системне ПЗ. 4. Прикладне програмне забезпечення – це програми, які виконують операції для конкретної прикладної задачі. 5. Мовами програмування є мови, якими програміст пише команди, що їх комп'ютер зрештою виконуватиме. 6. Мови низького рівня далі поділяються на машинну мову та мову

асемблера. 7. Термін “низький рівень” означає схожість із тим, як побудована машина. 8. Мови низького рівня є машинно-орієнтованими і вимагають широких знань комп’ютерного апаратного забезпечення та його конфігурації. 9. Машинна мова є єдиною мовою, яку комп’ютер розуміє безпосередньо. 10. Машинну мову вважають мовою першого покоління. 11. Мова асемблера стала першою спробою поліпшити структуру програмування. 12. Деякі комбінації літер можуть бути використані для того, щоб замінити собою числа машинного коду. 13. Мова асемблера має таку саму ефективність виконання, як і машинна мова, оскільки між програмою мовою асемблера та відповідною їй програмою машинною мовою існує однозначний транслятор. 14. Отже, зрозуміло, що програмування чи то машинною мовою, чи мовою асемблера є нелегкою справою. 15. Спочатку ФОРТРАН був дуже схожий на мову асемблера.

Exercise 2.9. Answer the questions.

1. How can a computer be made to work in a different manner?
2. What is computer software?
3. What does software provide a computer with?
4. How are computer software and hardware related? Explain this relationship.
5. How is the development of a large piece of software perceived?
6. What categories is computer software usually classified into?
7. What is application software? What may it consist of? Give some examples of application software.
8. What is a programming language? What two major types of programming languages exist?
9. What are the peculiarities of low-level languages? What does the term *low-level* mean?
10. What are the characteristic features of machine language? Why does machine language need no translator program?
11. What are the characteristic features of assembly language? In what way does it differ from machine language? What is the assembler?
12. What is a mnemonic opcode?
13. What were the reasons for the invention of high-level languages? What is the radical difference between high-level and low-level languages?
14. What were the languages FORTRAN and COBOL developed for?
15. What trend has been in the development of high-level languages since their introduction?

Exercise 2.10. Give derivatives of the following words and explain their meanings.

Control, sequence, change, complement, mean, complicate, develop, engine, perceive, care, rely, maintain, apply, analyze, instruct, execute, ultimate, direct, consider, express, increase.

Exercise 2.11. Give the opposites of the following words taken from text 2.1 and using them make up sentences of your own.

Meaningful; arise; care; construction; final; analysis; numerous; improve; save (*time, effort*); one-to-one; deep; instead of; increasingly; remote from.

Exercise 2.12. Find in text 2.1 the equivalents for:

emerge (appear), preassigned (predefined), in another way, order, to put it another way, enter, categorize (divide into, break down into), realize (consider, understand), profound,

eventually, part (element), perform, supply (give), main (principal, basic), of this kind, collection.

Exercise 2.13. Translate the sentences into English using as many equivalents of the underlined Ukrainian words as you can.

Model: Існує два **основні** види тестування програмного забезпечення: функціональне та структурне. – There are two **main (major, principal, basic)** types of software testing: functional and structural.

1. Практично всі розробники погоджуються з тим, що тестування програмного забезпечення є найменш зрозумілою складовою процесу розробки. 2. Одним із основних понять сучасної математики є поняття границі. 3. Поблочне тестування (unit testing) перевіряє окремі програмні компоненти або сукупність компонентів. 4. Всі історичні події в книзі було викладено в хронологічному порядку. 5. Для такої проблеми легкого рішення не існує. 6. Тестування програмного забезпечення класифікують відповідно до того, як виконуються перші два етапи процесу тестування. 7. Інакше кажучи, системи лінійних рівнянь зручно розв'язувати за допомогою матриць. 8. Відомо, що будь-яка хірургічна операція вимагає величезної обережності. 9. Цей код надає інформацію щодо внутрішніх параметрів, властивостей об'єкта тощо. 10. Студенти Національного авіаційного університету отримують глибокі знання з різноманітних предметів, необхідних у їхній майбутній професійній діяльності. 11. Користувач увів у комп'ютер комбінацію неперевіраних (untested) даних. 12. Системне тестування перевіряє сукупність елементів, що становить самостійний виріб. 13. Усі студенти нашого університету забезпечуються навчальною літературою. 14. Дві непаралельні прямі, що лежать в одній площині, врешті-решт перетнуться. 15. Фелікс почав усвідомлювати, що цю програму можна було зробити значно простішою.

Exercise 2.14. Study the vocabulary to text 2.2.

descendant – нащадок

imperative language – імперативна мова

functional language – функціональна мова

declarative language – декларативна мова

explicit – явний

to invoke – викликати, активувати (*програму, процедуру*)

data item – елемент даних

somewhat – почасти, деякою мірою

to supersede – витіснити

resolution – рішення

akin to – подібно до

pattern matching – зіставлення зі зразком; ототожнення
card reader – пристрій для читання карт
fault – несправність
compiler – компілятор
interpreter – інтерпретатор
utility program – утиліта
congested – перевантажений
diversified – різноманітний
management – керування
to compile – 1) компілювати; 2) укладати, упорядковувати
source program – початкова програма
routine – 1) підпрограма; 2) стандартна програма
sorting – сортування
report generation – генерація звітів
file updating – внесення змін до файла
file dump – роздрукування файла
file backup – створення резервної копії файла

Exercise 2.15. *Learn the following speech patterns.*

A

We **are to meet** at the station at six.

The lecture **was to begin** at eleven.

B

Professor Morrison has an advantage over his opponent **that** he is far more experienced

She could not find out anything about him **except that** he was a foreigner.

C

Approximately 8 netbooks **out of 10** run Windows operating systems now.

She asked only **out of curiosity**.

D

He wrote a program **whereby** he might solve that problem.

The man **whereof** he spoke is a professor of our university.

E

Microsoft's Windows is the most popular **PC-based** operating system.

This aircraft was created at the **Kyiv-based** Antonov Design Bureau.

Exercise 2.16. *Read and translate text 2.2.*

TEXT 2.2. TYPES OF HIGH-LEVEL LANGUAGES.

SYSTEM SOFTWARE

Types of high-level languages. COBOL, FORTRAN, and their descendants such as Pascal and C are known as *imperative* languages, since they specify as a sequence of explicit commands how the machine is to go about solving the problem at hand; this is not very different from what takes place at the machine level. Other languages are *functional*, in the sense that programming is done by calling (i.e. invoking) functions or procedures, which are sections of code executed within a program.

The best known language of this type is LISP (from List Processing), in which all computation is expressed as an application of a function to one or more "objects". Since LISP objects may be other functions as well as individual data items (variables, in mathematical terminology) or data structures, a programmer can create functions at the appropriate level of abstractions to solve the problem at hand. This feature has made LISP a popular language for artificial intelligence applications, although it has been somewhat superseded by logic programming languages as Prolog (from Programming in Logic). These are termed *nonprocedural*, or *declarative*, languages in the sense that the programmer specifies what goals are to be accomplished but not how specific methods are to be applied to attain those goals. Prolog is based on the concepts of resolution (akin to logical deduction) and unification (similar to pattern matching). Programs in such languages are written as a sequence of goals.

High-level languages have a major advantage over machine and assembly languages that they are easy to learn and use. It is because they are similar to the

languages we use in our everyday life. Their dis-advantage is that they require special software for translating high-level language programs into machine level programs.

System software exists to supervise the operation of the machine and to facilitate easy access to the various hardware resources. System software are general programs designed for performing tasks such as controlling all operations required to move data into and out of the computer. It communicates with printers, card readers, disks, tapes, etc., monitors the use of various hardware like memory, CPU etc.

Also, sys-tem software is essential for the development of applications software. System software allows application packages to be run on the computer with less time and effort. Remember that it is not possible to run appli-cation software without system software.

Development of system software is a complex task and it requires extensive knowledge of computer technology. Due to its complexity it is not developed in the house. Computer manufacturers build and supply this system software with the computer system. DOS, Unix, Windows are some of the widely used system software. Out of these Unix is a multi--user operating system whereas DOS and Windows are PC-based.

System software can be broken down into several categories:

1. The operating system: programs which supervise the operation of the entire system including controlling the execution of all other software and diagnosing faults.
2. Language processors: programs which translate application software written in high-level languages (e.g. Pascal) into the low-level binary machine instructions which the processor executes. There are three main types: compilers, assemblers, and interpreters.
3. Library and utility programs: software to help the application programmer. It consists of standard commonly used software routines which the user may invoke (e.g. for copying files of data).

An operating system is a set of programs to control and coordinate the operation of hardware and software in a computer system. It works analogously to a traffic police officer on a congested street: it provides simple, efficient, and reliable means for utilizing the many different parts of the computer. An operating system has a complex mixture of diversified functions. Its major functions are as follows:

- management of processing;
- memory management;
- input/output and file management;
- management of communications;
- security;
- user interface.

An assembler translates a program written in an assembly lan-guage into machine code. Each instruction in the assembly language program has almost a one-to-one correspondence to a machine instruction in the machine code. In other words, the opcode and operand that constitute each assembly instruction are expressed with a readable name

(i.e., a mnemonic opcode) and a decimal number, respectively, instead of the binary representation of the corresponding machine instruction. For example, LOAD 8 is an assembly instruction corresponding to the machine instruction 010100001000. Compared with other language processors, assemblers have relatively simple structures.

A *compiler* is a program translator that translates instructions of a high-level language to machine language. It is called compiler because it compiles machine language instructions for every program instruction of a high-level language. Thus, a compiler is a program translator like the assembler but more sophisticated. It scans the entire program first and then translates it into machine code.

An *interpreter* is another type of program translator used for translating high-level language programs into machine language programs. It takes one statement of a high-level language, translates it into machine language and immediately executes it. Translation and execution are carried out for each statement. It differs from a compiler, which translates the entire source program into machine code. Besides, a compiler is involved in the execution of a program.

A *program library* is a collection of available computer programs and routines.

Utility programs are specialized programs performing frequently required everyday tasks. Examples include sorting, report generation, file updating, file dump and backup, etc. Those programs are usually supplied by the manufacturer of the equipment.

Exercise 2.17. Find in text 2.2 the English for:

розробляти власними силами; в тому розумінні, що; взятись за розв'язання задачі; сприяти легкому доступу; вони називаються непроцедурними мовами; інтерфейс користувача; окремий елемент даних; досягти цілей; розв'язати наявну задачу; однозначна відповідність; пересилати дані в комп'ютер та з комп'ютера; контролювати роботу машини; контролювати використання різного апаратного забезпечення; з меншими витратами часу й зусиль.

Exercise 2.18. Translate into English paying special attention to the italicized words.

1. Мова асемблера є дуже зручною для комп'ютера, оскільки існує **однозначна відповідність** між її мнемокодами та командами в машинному коді. 2. Застосування комп'ютерів із більш досконалішими процесорами дозволяє **прискорити виконання** програм. 3. Матриця є чотирикутним масивом даних, в якому **елементи даних** розташовані у вигляді рядків та стовпців. 4. Одним із основних елементів операційної системи є **інтерфейс користувача**, завдання якого полягає в тому, щоб **полегшити спілкування** людини з комп'ютером. 5. Комп'ютери спеціального призначення **називаються** так, тому що вони є проблемно-орієнтованими, тобто спрямованими на розв'язання конкретної задачі. 6. Комп'ютерна програма є нічим іншим, ніж послідовністю команд, які вказують комп'ютеру, як **узятися за розв'язання даної задачі**. 7. Сучасні прикладні програми дозволяють розв'язувати найрізноманітніші задачі в багатьох сферах людської діяльності з **меншими**

витратами часу й зусиль. 8. Прикладне та системне програмне забезпечення доповнюють одне одне **в тому розумінні, що** виконання прикладної програми є неможливим без останнього. 9. Аби **досягти поставленої мети**, цей складний програмний продукт не слід розробляти **власними силами**. 10. Сортюванням масиву називається упорядкування **окремих елементів даних** цього масиву, здійснюване за тією чи іншою ознакою (збільшення або зменшення елементів масиву, відбір від'ємних або додатніх елементів тощо).

Exercise 2.19. *Say whether the following sentences are true or false. Correct the false ones.*

1. Pascal and C are predecessors of such languages as FORTRAN and COBOL. 2. LISP is a well-known example of declarative languages. 3. LISP and Pro-log were designed for artificial intelligence applications while FORTRAN - for solving mathematical problems. 4. Programs in declarative languages are written as a sequence of explicit commands. 5. System software is supplied by computer manufacturers separately from computer systems. 6. A compiler is involved in the execution of a program. 7. Among other things, an operating system is responsible for finding spelling errors during word processing as well as for fighting computer viruses. 8. System software is essential for the development of computer components. 9. An interpreter translates the entire source program into machine code. 10. Utility programs are specialized programs performing frequently required operations.

Exercise 2.20. *Complete the sentences translating their Ukrainian parts into English.*

1. COBOL, FORTRAN, and their descendants such as Pascal and C are known as *imperative* languages, (позаяк вони вказують у вигляді явних команд, як машина має взятися за розв'язання наявної задачі). 2. (Оскільки об'єктами LISP можуть бути інші функції, а також окремі елементи даних або структури даних), а programmer can create functions at the appropriate level of abstractions. 3. These languages are termed nonprocedural or declarative (в тому розумінні, що програміст вказує, які цілі мають бути досягнені, а не те, як слід застосовувати конкретні методи для досягнення цих цілей). 4. System software are general programs (які призначені для керування всіма операціями, необхідними для введення даних у комп'ютер та виведення з нього). 5. High-level languages have a major advantage over machine and assembly languages (в тому, що їх легко вчити і застосовувати). 6. System software communicates with printers, card readers, disks, tapes, etc., (контролює використання різного апаратного забезпечення, такого як пам'ять, процесор тощо). 7. Development of system software is a complex task (і воно вимагає широких знань обчислювальної техніки). 8. The operating system consists of programs which (контролюють роботу всієї системи, включаючи керування виконанням усіх інших програм і діагностику несправностей). 9. (Операційна система забезпечує прості, ефективні та надійні засоби) for utilizing the many different parts of the computer. 10. Each instruction in the assembly language program (має майже однозначну відповідність машинній команді у машинному коді). 11. (Інтерпретатор – це ще один вид транслятора) used for translating high-level language programs into machine language programs. 12. Utility programs are specialized programs (що виконують повсякденні завдання, в яких часто виникає потреба).

Exercise 2.21. *Translate into English.*

1. Інші мови є функціональними в тому розумінні, що програмування здійснюється викликанням функцій або процедур. 2. У мові LISP усі обчислення виражені як застосування функції до одного або більше об'єктів. 3. Ця особливість зробила LISP популярною мовою для прикладних задач штучного інтелекту, хоча її дещо потіснили мови логічного програмування. 4. Програми мовою Prolog пишуться у вигляді послідовності цілей. 5. Недоліком мов високого рівня є те, що вони вимагають спеціального програмного забезпечення для трансляції програм мовами високого рівня у програми машинного рівня. 6. Системне програмне забезпечення існує для того, щоб контролювати роботу машини та полегшувати доступ до різних апаратних ресурсів. 7. Системне програмне забезпечення робить можливим виконання на комп'ютері прикладних пакетів із економією часу і зусиль. 8. З огляду на його складність, системне програмне забезпечення не розробляють власними силами. 9. З-поміж цих систем Unix є багатокористувацькою операційною системою, тоді як DOS і Windows базуються на використанні ПК. 10. Порівняно з іншими трансляторами, асемблери мають відносно просту структуру. 11. Компілятор – це транслятор, подібний до асемблера, але складніший. 12. Інтерпретатор бере один оператор мови високого рівня, трансліює його в машинну мову й одразу виконує.

Exercise 2.22. *Answer the questions.*

1. What computer languages are called imperative? Give some examples. 2. What languages are referred to as functional? 3. How is computation in the language LISP expressed? 4. What may objects be in LISP? 5. What has made LISP a popular language for artificial intelligence applications? 6. What are the peculiarities of declarative languages? 7. What can be regarded as a disadvantage of high-level languages? 8. Why are high-level languages easier to learn and use as compared to machine and assembly languages? 9. What are the functions of system software? What sort of relationship is there between application software and system software? 10. Into what categories can system software be broken down? What are the functions of an operating system? 11. What is the purpose of language processors? Name their main types. 12. What are the peculiarities of an assembler? 13. What is the difference between a compiler and an interpreter? 14. What is the purpose of a program library and that of utility programs?

Exercise 2.23. *Give derivatives of the following words and explain their meanings.*

Sense, type, structure, solve, advantage, major, supervise, access, allow, time, supply, efficient, diversify, compare, entire, differ.

Exercise 2.24. *Give the opposites of the following words taken from text 2.2 and using them make up sentences of your own.*

Descendant, explicit, within, individual, variable, artificial, facilitate, essential, extensive, standard, congested, diversified, supply.

Exercise 2.25. *Speak on the following issues*

1. The types of high level languages.
2. The function of system software.
3. Classification of system software.

Grammar: Adjectives and Adverbs

Note 1. Adjectives are formed from nouns and verbs with the help of the suffixes: **-ful; -able/-ible; -ant/-ent; -ous/-ious; -ic; -ive; -y; -less; -al** and the negative prefixes **dis-; non-; un-; in-** (changed into **il-** before **l**, **ir-** before **r**, **im-** before **m** and **p**).

The comparative and superlative degrees are formed in two ways: 1) all one-syllable adjectives, adjectives of two syllables ending in **-y, -er, -le, -ow** form the degrees of comparison by means of the endings **-er** and **-est**; 2) other two-syllable adjectives as well as adjectives of three or more syllables form their degrees of comparison using the words **more** and **the most**. There is also a small group of adjectives (**good, bad, much, many, little, far**) that form the degrees of comparison in their individual way.

Exercise 2.26. *Change the following words into adjectives.*

Use, access, form, doubt, differ, patriot, science, progress, help, compare, equality, hero, danger, dirt, continue, legacy, vary, nature, value, negligence, honesty, resist, depend, act, aim, specify, physics.

Exercise 2.27. *Give the comparative and the superlative degree of the following adjectives.*

Modern, great, simple, extensive, good, small, large, complex, doubtful, high, flat, similar, effective, compact, fast, valid, regular, easy, early, extensive, new, versatile, personal, bad, accurate, little, useful, hard, natural, far, active.

Note 2. According to their meaning adverbs are subdivided into adverbs of time (today, yesterday, now, soon, early, late, then, before), adverbs of place (here, there, where, near, above), adverbs of repetition and frequency (often, never, sometimes, ever, usually, always), adverbs of degree (very, quite, little, too, much), adverbs of manner (quickly, well, badly, slowly, easily).

The most important suffix by means of which adverbs are formed from other parts of speech is the suffix **-ly**. Adverbs of direction have the suffix **-ward (s)**. Some adverbs have no suffixes (**hard, high, often, seldom**).

Adverbs have the same degrees of comparison as adjectives. Adverbs of one syllable and one two-syllable adverb (*early*) form their degrees of comparison by means of

the endings **–er, –est**. Adverbs of more than one syllable form their degrees of comparison by means of the words **more** and **most**.

Exercise 2.28. *Translate the following adverbs and give their comparative and superlative degrees.*

Рано, пізно, мало, скоро, наполегливо, повільно, швидко, добре, багато, погано, рідко, зручно, можливо, безпосередньо, першочергово, особливо, типово, скоро, легко, ймовірно, ясно.

Exercise 2.29. *Translate into English using adverbs.*

1. Інформація у двійковій формі обмежена лише двома значеннями. 2. Деякі комбінації літер можуть бути використані певним чином, щоб замінити собою числа машинного коду. 3. Якщо код займає багато пам'яті та працює дуже повільно, або кінцевий продукт може виконуватися лише однією операційною системою, це означає, що в ньому є певна кількість помилок. 4. Програма є спеціально впорядкованим списком команд. 5. DOS, Unix та Windows є системним програмним забезпеченням, що широко використовується в наш час. 6. Службові програми зазвичай постачає виробник разом із обладнанням. 7. Системне програмне забезпечення постачається виробниками комп'ютерів окремо від комп'ютерних систем. 8. Прикладне програмне забезпечення призначене спеціально для виконання певних, чітко визначених завдань.

Exercise 2.30. *Read the comparative constructions with adjectives and adverbs, translate them.*

as... as	1) This text is as difficult as that one. 2) The lessons today are as difficult as they were yesterday. 3) This electronic computation is as complex as that one. 4) Every scientific research is as important today as it was many years ago.
n times as ... as	1) 1) This notebook is twice as cheap as that one. 2) 2) Our department has twice as powerful computers as your department. 3) 3) This microprocessor is three times as compact as the previous one. 4) 4) My task is many times as hard as yours.
as ... as possible	1) The report must be written as well as possible . 2) This program should be as short as possible . 3) The microcomputers tend to be as fast as possible . 4) You have to do the job as quick as possible .

not so ... as, not as ... as	1) This job is not as good as that one. 2) My dictionary is not as good as yours. 3) This year you work at your English not so much as you did last year, do you? 4) High-level languages are not so difficult for me as assembly languages.
the ... the	1) The more I speak to you the more I like you. 2) The more I read your report the less I like it. 3) The older the computer the lower its processing speed. 4) The more people know about virtual reality, the better they control it.
than	1) She is a better programmer than her brother. 2) General-purpose computers are more versatile than special-purpose ones. 3) Workstations have greater memory than personal computers. 4) Laptops are more compact and they are better to transport than PCs.
much ... than	1) Our lecturer is much older than your lecturer is. 2) I have much more books on programming than you. 3) He understood the steps of program development much better than other students. 4) As a system analyst he has much more free time than I.
a bit ... than a little ... than	1) This programming language is a little easier than that one. 2) My notebook is a bit more expensive than my father's one. 3) The abilities of this device are a little higher than the abilities of the device developed by our competitors. 4) To find the error in this program was a bit more difficult than to debug it.

Exercise 2.31. *Change the following sentences according to the models.*

Model 1: Nick is a good programmer. Mike is a better programmer than Nick. – Nick is not as good a programmer as Mike (Nick is not such a good programmer as Mike).

Model 2: Nick is neither better nor worse than Mike. – Nick is as good as Mike.

1. Machine language has only two digits to operate. Assembly language has more digits to operate than machine language. 2. She studies neither more nor less subjects than her brother. 3. This is neither easier nor more difficult question than the previous one. 4. My program is neither longer nor shorter than his one. 5. I shall be busy tomorrow. Today I am busier. 6. This task is interesting. That task is more interesting. 7. Low-level language makes programmer's work convenient. High-level languages make programmer's work more convenient. 8. Operating system software is useful for computer processing. Application software is more useful for computer processing.

Exercise 2.32. *Decide whether the italicized words are adjectives or adverbs. Translate the sentences.*

1. This device works **properly**. 2. It provides **proper** execution of the instructions. 3. The questions were so **easy** that everybody could answer them. 4. He answered all the questions **easily**. 5. You would work **better** if you had a **better** computer. 6. He has very **little** knowledge of the subject. 7. He works **more** and **better** than he used to. 8. I have **more** books than you. 9. This program is performing **worse** than usual. 10. When I **first** came to Kyiv, I **little** thought that I should stay here so **long**.

Exercise 2.33. *Choose the right word from those in brackets.*

1. Application software is (general, generally) divided into two categories (horizontal, horizontally) and (vertically, vertical). 2. Because supercomputers can cost millions of dollars they are used (most, mostly) by government, scientific institutions and the military. 3. Computer software is (usual, usually) divided into application software and system software. 4. Now you can (hard, hardly) imagine the accountant's desk without a computer. 5. This is quite (clear, clearly). 6. My student wrote the program (bad, badly). 7. This (clear, clearly) shows the difference. 8. The description of the purpose of the program is not quite (exact, exactly). 9. The work was done (perfect, perfectly). 10. The explanation was quite (simple, simply). 11. The question can be answered quite (simple, simply). 12. Machine language is the only (suitable, suitably) language that is (direct, directly) understood by the computer.

Exercise 2.34. *Learn the words below, then read and translate text 2.3.*

cut across – охоплювати, включати

tailor (to sth) – пристосовувати (для кого-небудь/чого-небудь)

font – шрифт

page layout – компонування сторінки

paragraph indention – виділення абзацу

align – розташовувати, орієнтувати, вирівнювати

form letter – лист на бланку

ledgerlike format – формат бухгалтерської книги

entry – елемент (таблиці, списку, каталогу)

search – здійснювати пошук

dimensions – розміри

motion picture – фільм, кінозображення

fade away – поступово зникати

TEXT 2.3. KINDS OF APPLICATION SOFTWARE

Application software directs the computer to execute commands given by the user and includes any program that processes data for a user. Application software is generally divided into two groups: horizontal and vertical. A horizontal program can cut across many application areas. Vertical application programs are tailored to specific tasks. Application software includes word processors, spreadsheets, database management, inventory and payroll programs, design and manufacture applications, and many other kinds of “applications”. Popular software is usually distributed on magnetic disks and CDs. Modern software and hardware can handle text, graphics, sound, and movies. MS WORD, Lotus 1-2-3 and dBASE III Plus are examples of application software.

Text-editing programs. Letters, reports, and documents can be easily prepared on personal computers, because the user can see on the monitor what the text will look like when printed (abbreviated as WYSIWYG, which stands for “what you see is what you get” and is pronounced “wizzy-wig”). Programs are available that check spelling and grammar before printing. Because of progress in laser printer technology, text printed by a laser printer is of typeset quality. A wide range of fonts is now available, and their number is far greater than that available with conventional typeset printing.

All word processors offer facilities for document formatting, such as font changes, page layout, paragraph indention, and the like. Some word processors can also check spelling, find synonyms, incorporate graphics created with another program, correctly align mathematical formulas, create and print form letters.

Desktop publishing programs. Desktop publishing is the creation of pages for publication using personal computers. Utilizing programs for this purpose, one can easily draw pictures using a mouse as a pointer on the monitor of the computer. Text with arbitrary fonts can be easily mixed with the pictures, tables, and equations placed at desired locations.

Spreadsheet programs. Spreadsheet programs are the most popular programs for business calculations done on personal computers. They allow the user to enter columns

and rows of numbers in a ledgerlike format on a monitor. The user can define a formula relating these columns and rows, and then the results are automatically calculated as numbers are entered. When changes in some entries are made, the program recalculates the results based on this formula. For example, at midyear a company can estimate year-end profits by projecting different sales growth rates for its products.

Today's spreadsheets for PC's offer hundreds of functions. In addition, numerous spreadsheets representing different aspects of a business can be linked together. Changes in one cell of one spreadsheet are reflected in related cells in all linked spreadsheets.

Database management programs enable users to store large bodies of information and to search these databases in several ways.

Computer graphics programs enable computer users to create, change, and display pictures. The term *computer graphics* is also used to mean the pictures produced with these programs. The computer operator can create the original image on the computer or can use a previously created photograph or other picture that has been digitized. A digitized photo can be changed in a variety of ways. The user can change its dimensions or its colours, for example, or eliminate a part of it. Images in a photo can be moved or copied. Some computer graphics software works with motion pictures. Computer graphics plays a vital role in the publishing industry.

Presentation software is another major type of graphics program. This software enables users to create graphics to project onto motion-picture screens at meetings. Most presentation software can also produce special effects, such as images that fade away or are transformed into other images, and even sound effects.

Games software combines graphics, animation, sound, and music with clever design to produce exciting adventures and puzzles. Computer games are played on PC's or dedicated computers called video game units.

Virtual reality software uses graphics, sound, and other tools to create an artificial world through which a user can seem to move. Virtual worlds are filled with objects that can be "handled" by users wearing special sensor-lined gloves.

Computer-aided design (CAD) programs are essential to many professions.

Scientific visualization software is used in virtually every branch of science. One use of this software is development and testing of theories.

Exercise 2.35. *Find in text 2.3 the English for:*

текстовий редактор; друкарська якість; звичайний типографський друк; і тому подібне; графіка, створена іншою програмою; розташовані в потрібних місцях; формула, яка зв'язує ці стовпці й рядки; темпи зростання продажів; великі об'єми інформації; попередньо створена фотографія; відігравати важливу роль; кіноекран; начинені датчиками рукавиці.

Exercise 2.36. *Translate into English paying special attention to the italicized words.*

1. На виставці комп'ютерних технологій можна було побачити різні види сучасного прикладного програмного забезпечення, такого як програми керування базами даних, програми віртуальної реальності, ігрове програмне забезпечення, програми автоматизованого проектування *і тому подібне*. 2. Сучасні *текстові редактори* у поєднанні зі струменевими або лазерними принтерами забезпечують друк *друкарської якості*. 3. Електронні таблиці, де стовпці й рядки *зв'язані певними*

формулами, дуже зручні для різних розрахунків: математичних, економічних, статистичних **тощо**. 4. Програма Photoshop дозволяє радикально змінювати **попередньо створені фотозображення**. 5. Розумна цінова і маркетингова політика дозволила фірмі збільшити **темпи зростання продажів**. 6. Світ віртуальної реальності створюється за допомогою спеціальної гарнітури (eyephones), тривимірних окулярів (3D goggles) і **начинених датчиками рукавиць**, що дозволяють маніпулювати об'єктами, які бачить людина. 7. Складання прогнозів погоди на тривалий період потребує обробки **великих обсягів інформації**, тому метеослужби (meteorological services) багатьох країн використовують для цього суперкомп'ютери. 8. Застарілі **текстові редактори** не можуть мати в собі **графіку, створену іншою програмою**. 9. У наш час засоби автоматизованого проектування **відіграють важливу роль** у таких галузях техніки, як архітектура, машино-, автомобіле- та авіабудування **тощо**. 10. Під час автоматичного виготовлення друкованих плат (printed circuit boards) виводи елементів схеми вставляються в спеціально призначені отвори, **розташовані в потрібних місцях**.

Exercise 2.37. Translate the sentences paying attention to the meaning of the words in bold type.

1. Most **application** programs are now written in high-level languages. 2. The director received twenty **applications** for the position. 3. When you apply for a job, you must fill in an **application** form. 4. This liquid is for external **application** only. 5. Computer-aided design refers to any **application** of a computer to the solution of design problems. 6. You should remember that if you show **application** in your work, the boss may promote you soon. 7. This material **tailors** quite well. 8. This cloth is very difficult to **tailor**. 9. A **tailor-made** suit is usually much more expensive than a ready-made one. 10. The construction company **tailored** the house to the needs of the occupants. 11. All her novels are **tailored** to popular tastes. 12. A computer can **handle** numbers and letters. 13. It cost much effort to **handle** the situation. 14. This shop does not **handle** imported goods. 15. Oliver pulled the **handle** but the door proved to be locked on the inside. 16. Your indiscrete behaviour may give your enemies a **handle** against you.